Kunj Shah
CS 166 – Information Security

Evaluating Cross Architecture Security in Adversarial Neural Cryptography

Executive Summary

Adversarial neural cryptography is an experimental approach in which neural networks learn an encryption and decryption procedure through competition. An encrypting network, called Alice, and a decrypting network, called Bob, share a secret key. A third network, called Trudy, sees the ciphertext but not the key and tries to recover the original message. Alice and Bob are rewarded when Bob succeeds and Trudy fails. The concern is that Alice may learn only how to defeat the particular kind of Trudy used during training. This project will test that concern directly by training Alice and Bob against one attacker architecture, freezing them, and then training several new attacker architectures against the same learned cipher. The study will determine whether the apparent security transfers to attackers with different ways of learning patterns.

NOTE: The project evaluates neural cryptography as a research system; it does not claim to replace established cryptography such as AES or TLS.

1. Background Concepts

1.1 What Cryptography Does (You already know this)

Cryptography protects information by transforming a readable message, called plaintext, into an unreadable form, called ciphertext. A key controls the transformation. In symmetric encryption, the sender and receiver share the same secret key. The receiver uses that key to recover the original message. A secure system should allow the intended receiver to decode the message while preventing an attacker who sees the ciphertext from learning useful information about the plaintext.

Traditional encryption algorithms such as AES are designed by cryptographers and analyzed using mathematics, public review, and well-defined threat models. Adversarial neural cryptography is different. It asks whether neural networks can learn a transformation that behaves like encryption without being explicitly programmed with a known cipher. This makes it an interesting machine-learning experiment, but successful training alone is not proof of cryptographic security.

1.2 What a Neural Network Does (You may already know this)

A neural network is a parameterized function that learns from examples. During training, it receives an input, produces an output, measures how wrong that output is using a loss function, and adjusts its internal weights to reduce the error. Different network architectures learn different types of patterns. A convolutional neural network, or CNN, is naturally suited to local neighboring patterns. An LSTM is designed for sequential dependencies. A Transformer can directly model relationships between distant positions. A multilayer perceptron, or MLP, treats the input more generally as a fixed vector.

1.3 What Adversarial Training Means

Adversarial training means that two objectives compete. One model improves by defeating another model, while the opposing model improves in response. In this project, the word adversarial does not mean that someone manually attacks the computer during every training step. It means that Trudy is trained to recover the secret message, while Alice and Bob are trained to communicate in a way that

makes Trudy fail. As Trudy improves, Alice and Bob must also improve. This creates a repeating contest.

This setup resembles the competitive idea used in generative adversarial networks, but the roles are different. Alice is an encryptor, Bob is a legitimate decryptor, and Trudy is an eavesdropper. Trudy may be trained as a message-reconstruction model or, in stronger threat models, as a classifier that distinguishes encryptions of selected messages.

2. What Adversarial Neural Cryptography (ANC) Is

The best-known setup was introduced by Abadi and Andersen in 2016 [1]. It contains three trainable neural networks: Alice, Bob, and Trudy. Alice receives the plaintext message M and secret key K. Alice outputs ciphertext C. Bob receives C and K and attempts to reconstruct M. Trudy receives only C and attempts to reconstruct M without the key.

$$\begin{array}{l} M + K \rightarrow \text{ Alice } \rightarrow C \rightarrow \text{ Bob} + K \rightarrow M \\ \qquad\qquad\qquad\qquad\quad \downarrow \\ \qquad\qquad\qquad\qquad\quad \text{Trudy} \rightarrow M' \end{array}$$

Training alternates between the participants. Trudy is trained to reduce its error when guessing the plaintext. Alice and Bob are trained to keep Bob's reconstruction error low while pushing Trudy toward random guessing. For a balanced binary message, random guessing produces approximately 50 percent bit accuracy. Therefore, a useful run has two simultaneous outcomes: Bob is close to 100 percent correct, while Trudy remains close to 50 percent.

When we say that Alice and Bob are trained against a CNN Trudy, the CNN is not a prebuilt attack obtained from somewhere else. We create the Trudy with CNN architecture with random weights and train it as part of the experiment. It receives ciphertext as input and uses the known plaintext during training to calculate its error. It never receives the secret key. This corresponds to a learning-based known-plaintext setting: the attacker can study example plaintext/ciphertext pairs and must generalize to new ciphertexts.

3. The Security Evaluation Problem

Suppose Alice and Bob are trained only against a CNN Trudy. During training, the CNN may fail, so the system appears secure. However, that result proves only that the trained CNN could not recover the message under the chosen training conditions. Alice may have hidden local patterns that a CNN recognizes while accidentally leaving a different global or sequential pattern in the ciphertext. An LSTM or Transformer may exploit that remaining pattern.

The central distinction is between defeating one attacker and learning a transformation that remains difficult for many reasonable attackers. This is similar to preparing for one exact exam rather than learning the entire subject. A system can score well against the attacker it repeatedly sees while still failing against a new attacker with a different learning bias.

Our term cross architecture means that the attacker architecture used after training is different from the attacker architecture used while Alice and Bob learned. The attack task does not change: every Trudy sees ciphertext, receives no key, and tries to recover plaintext. Only the model architecture changes. This controlled comparison lets us ask whether the learned cipher overfits to a particular kind of attacker.

4. What Previous Publications Have Done

The literature already covers important pieces of this problem. The proposed work must therefore be described narrowly. It is not enough to say that we will use a stronger Trudy, retrain Trudy, compare neural architectures, or train against multiple attackers; versions of those ideas already exist.

4.1 Foundation and Stronger Threat Models

Abadi and Andersen [1] introduced the modern Alice–Bob–Trudy framework and showed that neural networks could learn transformations that allowed Bob to recover the message while reducing Trudy's accuracy. They also reset and retrained Trudy after Alice and Bob were frozen, which is important because it checks whether the first Trudy simply trained poorly. However, the new Trudys used essentially the same architecture. Their evaluation tests new initialization and retraining, not systematic transfer across different attacker architectures.

Coutinho and colleagues [2] argued that message reconstruction by Trudy is not a sufficiently strong definition of security. They introduced a chosen-plaintext indistinguishability-style task in which Trudy attempts to determine which of two messages produced a ciphertext. This changes both Trudy's objective and the loss function and moves the experiment closer to conventional cryptographic reasoning.

Zhou and colleagues [3] analyzed weaknesses in the original intelligent symmetric-key model. Their work examined ciphertext properties, plaintext and key leakage, key-recovery attackers, and multiple adversarial models. It shows that an apparently unsuccessful reconstruction Trudy does not establish broad security. Li and colleagues [4] later proposed the Adversarial Network Encryption System, or ANES, with different attacker capabilities, a general loss formulation, and multiple attackers. They reported that stronger and more numerous opponents increased the chance of learning one-time-pad-like behavior.

4.2 Architecture and Scale Changes

Zhu, Vargas, and Sakurai [5] investigated topology-evolving neural networks. Their work demonstrates that neural cryptography is not restricted to the original fixed CNN design. Duan and colleagues [6] modified architecture and loss design using ideas associated with deep convolutional adversarial networks and explored longer messages. These studies change the overall neural cryptography system, but they do not isolate the question of whether a cipher learned against attacker architecture A transfers securely to independently trained attacker architecture B.

Tyler, Fadul, and Reising [8] compared CNN, LSTM, and their RANDOM architecture as complete adversarial neural cryptography systems. That comparison is highly relevant because it demonstrates that architecture affects the design and behavior of the system. However, comparing CNN Alice/Bob/Trudy with LSTM Alice/Bob/Trudy is not the same as freezing one Alice/Bob pair and attacking it with CNN, LSTM, MLP, and Transformer Trudys under matched conditions. The latter cross-architecture matrix is the focus of this proposal.

4.3 Expanded Applications and Recent Reviews

Meraouche and colleagues [7] extended adversarial neural cryptography toward asymmetric encryption with public/private-key-generating networks and evaluated stronger adversaries, including leakage and chosen-plaintext-style attacks. Wang and colleagues [9] proposed AdvOT, an adversarial-network approach to oblivious transfer with multiple attackers. These works reinforce that multiple or stronger adversaries are established research directions. A 2026 survey by Zhong, Gu, and Yu [10] describes neural cryptography as a field covering both constructive uses of neural networks in cryptographic systems and neural-network-based cryptanalysis, while emphasizing unresolved evaluation and security challenges.

Based on the literature reviewed for this proposal, I have not found a study whose primary controlled experiment is: train against attacker architecture A, freeze Alice and Bob, train attacker architectures A, B, C, and D independently under matched budgets, and repeat across all training architectures. This is a provisional literature finding, not a guaranteed novelty claim. A formal citation-chain and database review would still be required before publication. Three additional factors that could be added can be scale of the Alice and Bob compared to the attackers, the method of Syncing Alice-Bob networks and finally the size of the key and plaintext.

5. Proposed Research

5.1 Research Question

To what extent does adversarial neural cryptography overfit to the architecture of the attacker used during training? More specifically: does an Alice–Bob system trained against one Trudy architecture remain resistant when fresh Trudys with different architectures are optimized against the frozen learned cipher?

5.2 Core Experiment

1. **Train the communication system.** Choose one training Trudy architecture—MLP, CNN, LSTM, or Transformer. Train Alice, Bob, and that Trudy adversarially until Bob reliably reconstructs the message and the training Trudy no longer improves.
2. **Freeze Alice and Bob.** After training, stop all updates to Alice and Bob. This prevents the cryptosystem from adapting when a new attacker is introduced.
3. **Create fresh attackers.** Initialize new MLP, CNN, LSTM, and Transformer Trudys from random weights. Generate plaintext, key, and ciphertext examples using frozen Alice. Each new Trudy receives ciphertext as input, plaintext as its training target, and no key.
4. **Train every attacker independently.** Give each Trudy a matched training-data budget, parameter budget, optimizer policy, and stopping rule. Train several random seeds so that one lucky or unlucky initialization does not determine the conclusion.
5. **Repeat the full procedure.** Train separate Alice–Bob systems against each of the four Trudy architectures, then attack every frozen system with all four fresh Trudy architectures. This produces a 4 × 4 result matrix.

Training Trudy	MLP Attack	CNN Attack	LSTM Attack	Transformer Attack	SSM Attacks
MLP	?	?	?	?	?
CNN	?	?	?	?	?
LSTM	?	?	?	?	?
Transformer	?	?	?	?	?
SSMs	?	?	?	?	?

5.3 Controls and Measurements

- **Communication quality.** Measure Bob's bit accuracy, full-message accuracy, and Hamming error. A system is not useful if Bob cannot reliably decode.
- **Attack success.** Measure each Trudy's bit accuracy, full-message recovery rate, Hamming error, and advantage above random guessing on unseen examples.

- **Matched comparisons.** Keep message length, key length, dataset size, training steps, optimizer search, and approximate parameter count comparable across architectures.
- **Repeated runs.** Use at least five random seeds per condition and report mean, standard deviation, confidence intervals, and the best post-training attacker.
- **Sanity baselines.** Include a deliberately weak transform that should be learnable by Trudy and a one-time-pad-style baseline that should not rTrudyal plaintext without the key.

6. Possible Results and Conclusions

- **New architectures break the cipher.** If the training Trudy stays near random guessing but a different Trudy recovers substantially more plaintext, the system exhibits adversary overfitting. The learned transformation defeated one attack strategy rather than the general task.
- **All reasonable Trudys fail.** If all fresh attackers remain near random guessing while Bob remains accurate, the system shows empirical cross-architecture robustness. This is stronger evidence than testing one Trudy, but it is still not a mathematical proof of security.
- **Some architectures succeed and others fail.** This result would show that security depends on architectural inductive bias or capacity. It would identify which model families are more effective for neural cryptanalysis and could guide future evaluation standards.
- **Ensemble training improves transfer.** As an optional second phase, Alice and Bob could be trained against multiple architecturally different Trudys and then tested against an unseen architecture. This would test whether attacker diversity reduces overfitting.

7. Expected Contribution, Feasibility, and Limitations

The main contribution would be a reproducible PyTorch benchmark for cross-architecture security evaluation in adversarial neural cryptography. The benchmark would separate architecture effects from parameter count and training budget, report a complete attack-transfer matrix, and provide a stronger evaluation procedure than relying on one training Trudy. The project is feasible for undergraduate research because the initial experiments can use short binary messages and modest neural networks rather than large language models or large datasets.

The project has important limitations. Failure by the tested Trudys does not prove cryptographic security; a future attacker, analytical method, or larger model may still succeed. Results may depend on message length, training stability, hyperparameter search, and the assumed known-plaintext data. The first phase should therefore focus on a clean architecture comparison. Loss-function transfer, ensembles, chosen-plaintext objectives, and larger messages should be treated as later extensions rather than mixed into the initial experiment.

8. Proposed First Milestones

1. **Reproduce a baseline.** Implement the original small Alice–Bob–Trudy symmetric setup and confirm that Bob learns while Trudy approaches random guessing.
2. **Build the attack harness.** Freeze Alice and Bob, generate post-training attack data, and verify that a fresh same-architecture Trudy can be trained correctly.
3. **Run a pilot matrix.** Start with CNN and Transformer Trudys using matched capacities and five seeds. If the pipeline is stable, add MLP and LSTM to complete the 4×4 matrix.

References

[1] M. Abadi and D. G. Andersen, 2016. Learning to Protect Communications with Adversarial Neural Cryptography.

[2] M. Coutinho, R. de Oliveira Albuquerque, F. Borges, L. J. García Villalba, and T.-H. Kim, Sensors, 2018. Learning Perfectly Secure Cryptography to Protect Communications with Adversarial Neural Cryptography.

[3] L. Zhou, J. Chen, Y. Zhang, C. Su, and M. A. James, Computers and Security, 2019. Security Analysis and New Models on the Intelligent Symmetric Key Encryption.

[4] Z. Li, X. Yang, K. Shen, R. Zhu, and J. Jiang, Neurocomputing, 2020. Information Encryption Communication System Based on the Adversarial Networks Foundation.

[5] Y. Zhu, D. V. Vargas, and K. Sakurai, CANDAR Workshops, 2018. Neural Cryptography Based on the Topology Evolving Neural Networks.

[6] X. Duan, Y. Han, C. Wang, and X. Bi, IEEE BDAI, 2021. Optimization of Encrypted Communication Length Based on Generative Adversarial Network.

[7] I. Meraouche, S. Dutta, H. Tan, and K. Sakurai, Engineering Applications of Artificial Intelligence, 2023. Learning Asymmetric Encryption Using Adversarial Neural Networks.

[8] J. H. Tyler, M. K. M. Fadul, and D. R. Reising, Machine Learning with Applications, 2025. Transformers—Messages in Disguise.

[9] Y. Wang, Z. Zhong, N. Liu, X. Li, and J. Wang, Neurocomputing, 2025. AdvOT: Oblivious Transfer Based on Generative Adversarial Networks Against Multiple Attackers.

[10] Y. Zhong, J. Gu, and X. Yu, Neurocomputing, 2026. Neural Cryptography: Synergy and Conflict Between Neural Networks and Cryptographic Systems.